03 — Database Design and Implementation

Item	Details
Course	DBMS Project — Semester 5
Team	Sneha Angelin Jisho (PES2UG24CS507); Sowmya Ramesh (PES2UG24CS512)
Stack	React + Vite; Node.js + Express; MySQL; MongoDB
Repository	https://github.com/snehajisho/Satellite_Data_Provenance_Earth_Observatory-DBMS_Project

7. Database Choice and Justification

7.1 MySQL

MySQL stores structured records with explicit relationships and integrity requirements. It is used for missions, satellites, satellite–mission assignments, sensors, ground stations, geographic locations, observations, data products, processing activities and application users. Primary keys, foreign keys, unique constraints and CHECK constraints represent relationships and domain rules.

7.2 MongoDB

MongoDB stores documents in the `observation_metadata` collection. Each document groups observation identifiers with nested satellite, sensor, location, image, spectral, quality and processing metadata. This document model accommodates nested and evolving metadata without requiring every optional attribute to become a relational column.

7.3 Combined use

The databases are complementary, not replicas. Both are named `satellite_geo_repository`, but they are separate database systems. MySQL is accessed through `mysql2/promise`; MongoDB is accessed through the MongoDB Node.js driver. API routes read from the relevant store. The implementation does not provide automatic replication or distributed cross-database transactions.

8. Data Dictionary / Schema Description

8.1 Relational tables

Table	Attributes (PK/FK noted)	Purpose
MISSION	mission_id (PK), mission_name, agency, country, launch_date, start_date, end_date, status, description	Mission identity, agency, dates and status.
SATELLITE	satellite_id (PK), satellite_name (UNIQUE), launch_date, orbit_type, orbit_altitude_km, status	Satellite identity and orbit details.
SATELLITE_MISSION	mission_id (PK/FK), satellite_id (PK/FK), assignment_start, assignment_end, role	Associative table for mission–satellite assignments; composite PK.
SENSOR	sensor_id (PK), satellite_id (FK), sensor_name, sensor_type, spatial_res, spectral_range, status	Sensors associated with satellites.
GROUND_STATION	ground_station_id (PK), station_name (UNIQUE), operator, country, latitude, longitude, altitude, status	Ground-station sample records and coordinates.
GEO_LOCATION	geo_location_id (PK), area_name (UNIQUE), country, region, min/max latitude, min/max longitude	Geographic areas and bounding coordinates.
OBSERVATION	observation_id (PK), sensor_id (FK), ground_station_id (FK), geo_location_id (FK), observation_start/end, orbit_number, cloud_cover_percent, observation_status	Observation event details and relationships.
DATA_PRODUCT	product_id (PK), observation_id (FK), geo_location_id (FK), product_name, product_type, processing_level, version, creation_date	Products associated with observations and locations.
PROCESSING_ACTIVITY	activity_id (PK), input_product_id (FK), output_product_id (FK), processing_type, software_name, parameters, started_at, completed_at, status	Processing lineage between input and output products.
USERS	user_id (PK), username, email (UNIQUE), password_hash, role, created_at, is_active	Application accounts and roles.

8.2 Integrity constraints

- Mission dates are checked so end_date is not earlier than start_date and start_date is not earlier than launch_date.
- Satellite orbit altitude and sensor spatial resolution must be positive when provided.

- Ground-station coordinates and geographic bounding coordinates are range-checked; minimum bounds cannot exceed maximum bounds.
- Observation end time cannot precede start time; orbit number must be positive; cloud cover must be 0–100.
- Product name/version is unique; processing input and output products must differ; completion time cannot precede start time.

8.3 MongoDB document structure

Collection: `observation_metadata` in database `satellite_geo_repository`; sample data script creates one document per observation.

Top-level identifiers include `observation_id` and `data_product_id`.

Nested groups:

- `satellite`
- `sensor`
- `location`
- `image_metadata`
- `spectral_metadata`
- `quality_metadata`
- `processing_metadata`

Image metadata contains file formats, thumbnail/image paths, supported visualizations and source product information. Indexes are created on observation_id, cloud-cover metadata and satellite ID.

9. SQL/NoSQL Implementation

9.1 Sample dataset

The sample population script contains 3 missions, 4 satellites, 4 satellite–mission assignments, 5 sensors, 3 fictional sample ground stations, 10 geographic locations, 20 observations, 20 data products and 10 processing activities. The USERS table is defined in the schema script.

9.2 Views

View	Purpose
vw_observation_details	Joins observations with satellite, sensor and geographic location details.

View	Purpose
vw_data_product_details	Joins data products with their observation and location.
vw_processing_provenance	Shows processing activities with input and output product names.

9.3 Indexes

The index script creates indexes on OBSERVATION(sensor_id), OBSERVATION(geo_location_id), OBSERVATION(observation_start), DATA_PRODUCT(observation_id), and PROCESSING_ACTIVITY(input_product_id) and PROCESSING_ACTIVITY(output_product_id).

9.4 Stored procedures and function

Database object	Type	Purpose
GetObservationsBySatellite(p_satellite_id)	Stored procedure	Returns observations for a satellite with sensor, location, time, cloud cover and status.
GetProductsByLocation(p_geo_location_id)	Stored procedure	Returns products for a geographic location.
CountObservationsBySatellite(p_satellite_id)	Function	Returns the number of observations associated with a satellite.

9.5 Triggers

Trigger	Timing	Rule
trg_validate_cloud_cover_insert	BEFORE INSERT on OBSERVATION	Rejects cloud cover below 0 or above 100.
trg_validate_cloud_cover_update	BEFORE UPDATE on OBSERVATION	Rejects cloud cover below 0 or above 100.
trg_validate_processing_activity	BEFORE INSERT on PROCESSING_ACTIVITY	Rejects completion time earlier than start time.

9.6 MongoDB population

`database/mongodb/data.js` selects `satellite_geo_repository`, drops and repopulates `observation_metadata`, creates indexes and prints document-count/index information. Because it drops the collection before inserting, it is a reseed script; running it replaces the existing collection contents.

9.7 Backend connectivity and authentication

- `backend/db.js` creates a MySQL promise pool from DB_HOST, DB_PORT, DB_USER, DB_PASSWORD and DB_NAME.
- `backend/mongo.js` connects through MONGO_URI and selects MONGO_DB_NAME using the MongoDB Node.js driver.
- Registration hashes passwords using bcrypt. Login compares the supplied password with the stored hash and signs a JWT containing user_id and role; expiry is two hours.
- `requireAuth` verifies a Bearer token; `requireAdmin` checks that the decoded role is ADMIN.